Điểm danh - Học viên Chương trình Đào tạo Nhân tài AI thực chiến

Phiếu khảo sát cho Ngày 1 Khai giảng và Học nội dung: AI & LLM Foundation

AI IN ACTION-P1 · NGÀY 2

Xác định bài toán cho AI

Từ brief mơ hồ → Problem Statement rõ ràng

VinUniversity · Phase 1 · Tuần 1 · 2026

Instructor

Mai Anh Nguyen (Blue)

Generalist Product Builder

- 2026 FPT Long Châu (PM · Healthcare Product)
- 2025 Thongtincuuho.org (Co-founder)
- 2025 FPT Software AI Center (PM · AI Agent)
- 2021–2025 Xantus (PM · On-chain Analytics, AI Agent)
- 2016–2021 DYNO, Kalapa (PM · OCR, eKYC, Credit Scoring)

Interest: applied AI in enterprise productivity, human-AI experience design, AI education.

Hôm nay bạn sẽ học gì?

3 câu hỏi xuyên suốt ngày:

1. Có thật sự nên dùng AI không?
2. Nếu có, nên dùng mức nào: rule, feature, hay agent?
3. Khi nào nên Go / Not Yet / No-Go?

Sáng: học framework + case study → **Chiều:** tự làm với bài toán thật từ chính bạn

Flow buổi học

SÁNG — LECTURE (3H)

— Có nên dùng AI không? Buy vs Build

— Design Thinking + Human-Centered Design

— AI System + 5 Workflow Patterns

— AI Readiness Checklist + Problem Statement

— Go / Not Yet / No-Go

CHIỀU — LAB (3.5H)

👤 **Cá nhân:** Scan 5+ bài toán từ chính mình

👤 **Cá nhân:** Viết Quick Problem Card top 3

👥 **Nhóm:** Pitch – Challenge – Vote → chọn 1

👥 **Nhóm:** Deep-Dive: workflow, PS, research

👥 **Nhóm:** Evaluate: AI Readiness → Go/No-Go

👤 **Cá nhân:** Reflection Log

DELIVERABLES

➤ **Deliverable 1**

Scan & Filter Log (cá nhân)

➤ **Deliverable 2**

Problem Deep-Dive (nhóm)

➤ **Deliverable 3**

Reflection Log (cá nhân)

Luật chơi buổi sáng

Tương tác = học được nhiều hơn

💬 **Discord là kênh chính**

Bài tập, câu trả lời, thảo luận — gửi lên Discord

🙋 **Tham gia, không chỉ ngồi nghe**

Trả lời câu hỏi, comment bài bạn khác, chia sẻ góc nhìn

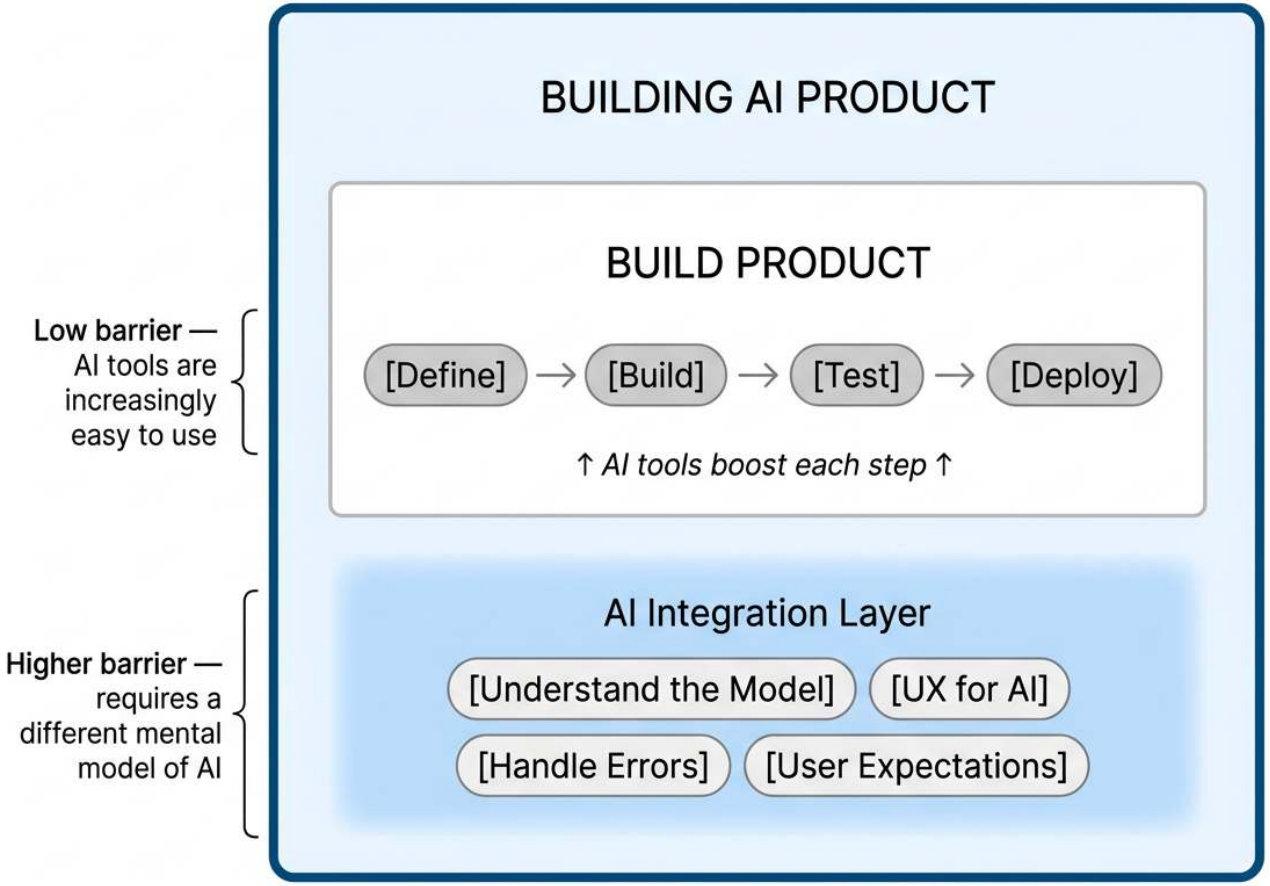
✅ **Không có câu trả lời sai**

Chỉ có câu trả lời chưa đủ sâu — và đó là cơ hội học

🎁 **5 bạn tương tác tích cực nhất buổi sáng → Perplexity Pro 1 năm**

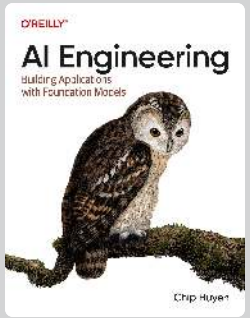
Building AI Product

AI Product chứa Build Product bên trong — không thay thế



Một số cuốn sách nền tảng

Deep-dive thêm sau khóa học

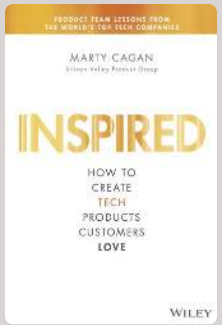


KỸ THUẬT & HỆ THỐNG

AI Engineering

Chip Huyen (2025)

"Có API rồi — làm gì tiếp?" RAG, agents, guardrails, eval, production ops.

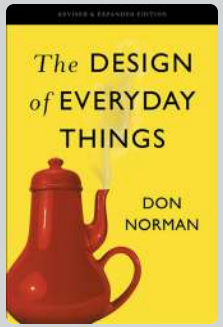


PRODUCT & STRATEGY

Inspired

Marty Cagan (2018)

Build cái gì, cho ai, và tại sao họ cần? Khám phá vấn đề thay vì feature factory.



TƯ DUY & THIẾT KẾ

Design of Everyday Things

Don Norman (2013)

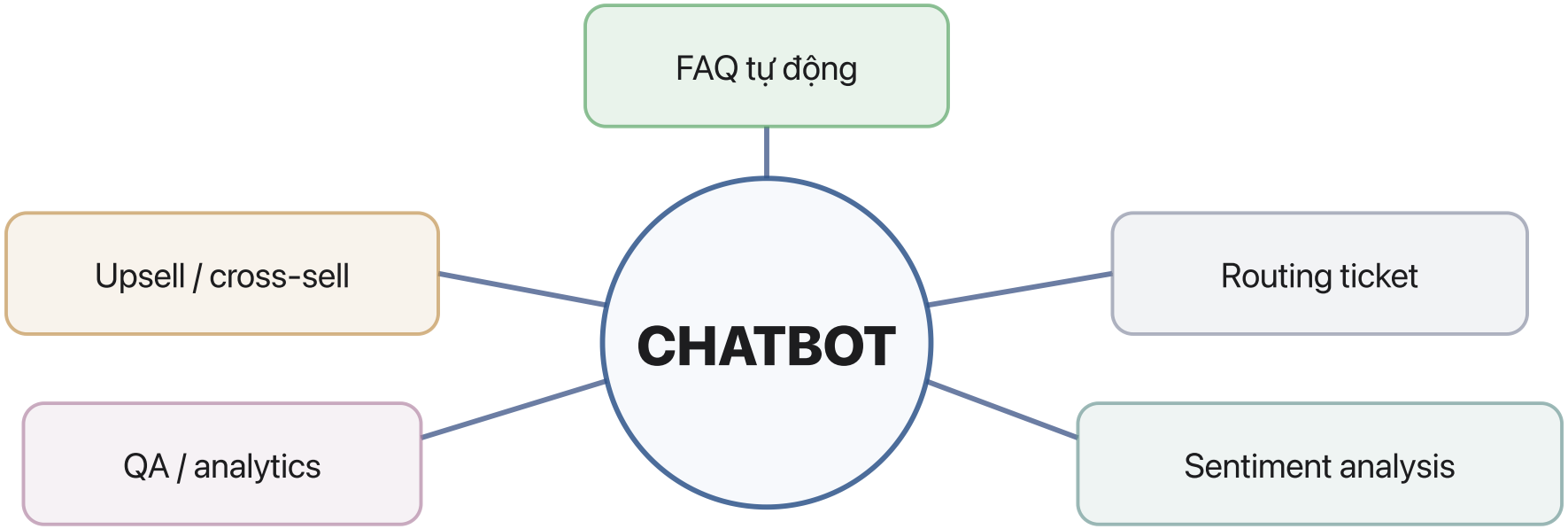
Affordance, feedback, mental model. Nền tảng UX — đặc biệt quan trọng khi AI không giải thích được.

“Tôi muốn xây chatbot cho khách hàng”

💬 Pause & Think

Nếu stakeholder nói "Tôi muốn làm chatbot", theo bạn họ thực ra đang muốn giải bài toán gì?

"Chatbot" không phải 1 bài toán



1 brief = 5 bài toán = 5 kiến trúc = 5 metric khác nhau

💬 Tình huống

Lớp 500 người, 10 Lab Coach, buổi lab chiều

ai cũng cần hỗ trợ nhưng không đủ người.

Dùng AI giải quyết thế nào?

Viết câu trả lời của bạn · Gửi lên Discord · 5 phút

Khoan đã — bạn có hỏi không?

Trước khi nghĩ solution, hãy hỏi đúng câu hỏi

🔍 Bạn có hỏi: **500 SV đang gặp khó ở chỗ nào** không?

🔍 Bạn có hỏi: **10 Lab Coach hiện kẹt ở đâu** không?

🔍 Bạn có hỏi: **hiện tại giải quyết bằng cách nào rồi** không?

Pain points thực sự là gì?

✍️ Bài tập cá nhân

Nghĩ lại Day 1.

Viết ít nhất **3 pain points** bạn trải nghiệm / quan sát được,

và muốn nó tốt hơn.

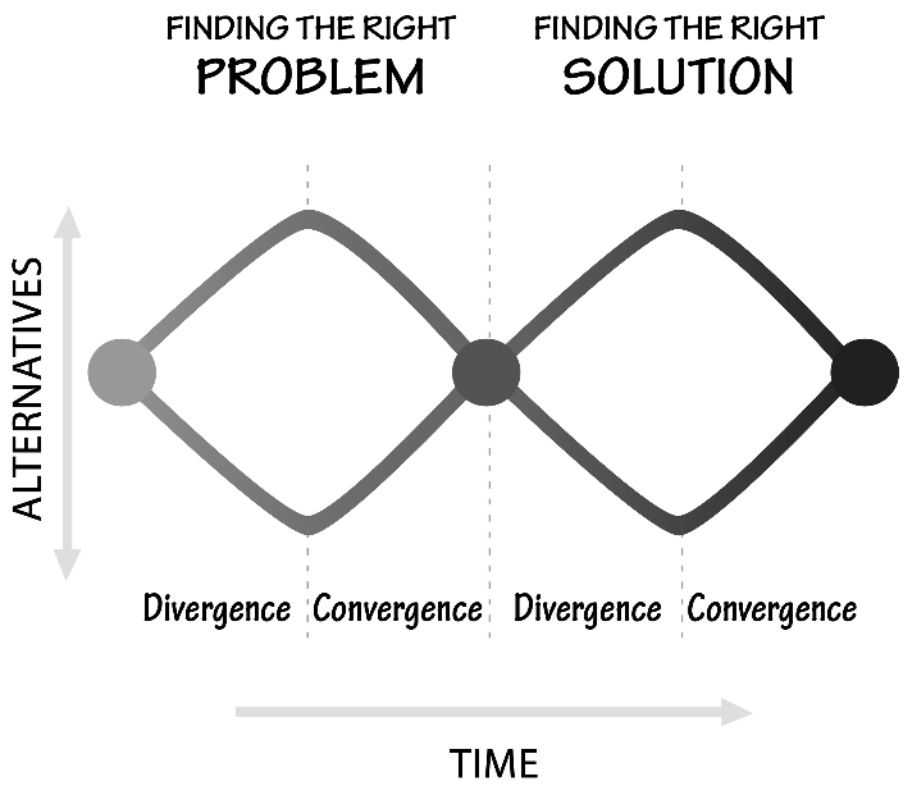
Gửi lên Discord · 5 phút

“Đừng bao giờ giải quyết vấn đề mà bạn được yêu cầu giải quyết.”

Don Norman — *The Design of Everyday Things* (2013)

Tìm đúng vấn đề trước khi tìm giải pháp

Mô hình Double Diamond — Don Norman / British Design Council (2005)



Diamond 1 — Tìm đúng vấn đề

Discover: Mở rộng — khảo sát vấn đề căn bản

Define: Thu hẹp — xác định đúng bài toán gốc

Diamond 2 — Tìm đúng giải pháp

Develop: Mở rộng — nhiều giải pháp tiềm năng

Deliver: Thu hẹp — chọn và triển khai

*Kỹ sư và doanh nhân được đào tạo để **giải** vấn đề.*
*Nhà thiết kế được đào tạo để **khám phá** vấn đề thật.*

Giải pháp xuất sắc cho sai vấn đề có thể còn tệ hơn không có giải pháp.

Kim cương 1 — Tìm đúng vấn đề

Các kỹ thuật cụ thể cho Discover và Define

DISCOVER (MỞ RỘNG)

- Observation / Field Visit
- User Interview
- Survey
- Diary Study
- Data Mining / Log Analysis
- Stakeholder Mapping

Nguyên tắc: Không judge, không filter.
Ghi hết. Nhiều nguồn. Nhiều góc nhìn.

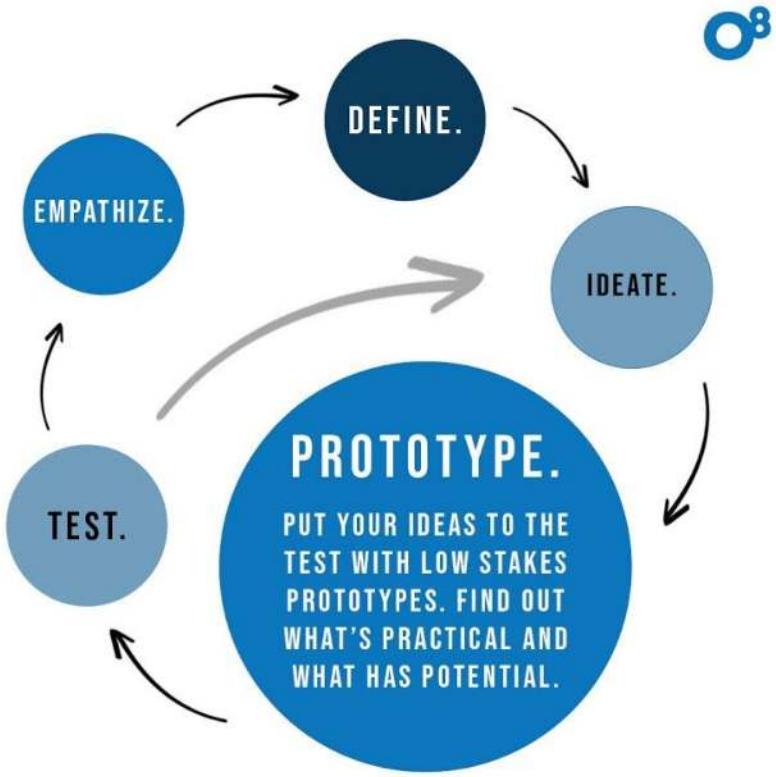
DEFINE (THU HẸP)

- Affinity Mapping
- How Might We
- Impact-Effort Matrix
- Dot Voting
- Problem Statement
- 5 Whys

Nguyên tắc: Gom trùng. Ưu tiên bằng data
(bao nhiêu người đau? đau bao nhiêu?).
Ra 1 vấn đề rõ, đo được.

Quy trình thiết kế lấy con người làm trung tâm (HCD)

4 bước lặp lại bên trong mỗi Diamond — Don Norman



Observation (Quan sát)

Những người được quan sát phải phù hợp với đối tượng mục tiêu.

Quan sát những khách hàng tiềm năng trong cuộc sống bình thường, hiểu các tình huống thực tế mà họ gặp phải.

Ideation (Tạo ra ý tưởng)

Tạo ra nhiều ý tưởng, sáng tạo mà không bị ràng buộc bởi các hạn chế.

Tránh phê bình ý tưởng của bản thân hay của người khác.

Đặt câu hỏi về tất cả mọi thứ.

Prototype (Tạo mẫu thử)

Tạo một nguyên mẫu nhanh cho mỗi giải pháp tiềm năng.

Mục tiêu là kiểm tra nhanh nhất có thể trước khi build.

Test (Kiểm tra)

Ngồi quan sát cách người dùng tương tác với Prototype trong thực tế.

Iteration (Lặp lại)

Tinh chỉnh và nâng cao liên tục.

Những câu hỏi "ngớ ngẩn" đã thay đổi thế giới

Norman: "Hãy đặt câu hỏi về tất cả mọi thứ — đặc biệt là những điều hiển nhiên"

"Nếu quá táo rơi xuống, thì mặt trăng có rơi không?"

Isaac Newton

Câu hỏi ngớ ngẩn về quả táo → nền tảng toàn bộ cơ học cổ điển & cách mạng công nghiệp.

"Tại sao con phải đợi lâu mới xem được ảnh?"

Jennifer Land, 3 tuổi (1943)

Câu hỏi của đứa trẻ → cha cô bé phát minh máy ảnh chụp lấy ngay Polaroid.

"Nếu cho thuê phòng trống cho khách du lịch thì sao?"

Brian Chesky & Joe Gebbia (2007)

Hai designer vật lộn tiền thuê nhà → Airbnb, định giá hơn 100 tỷ USD.

✍️ Bài tập cá nhân

Bạn có câu hỏi nào

mà cảm thấy **“ngớ ngẩn”** không?

Viết lên Discord · 3 phút

Problem-first, not AI-first

3 case studies thực tế

Team Cursor

Xây AI cho ngành mình không hiểu sâu (4 tháng trong cơ khí) → thiếu dữ liệu, thiếu insight domain.
Kết quả: Hủy ý tưởng, pivot về phần mềm.

Artifact

AI cá nhân hóa tin tức tốt, nhưng market dynamics yếu (thiếu viral loop tự nhiên).
Kết quả: Đóng cửa sản phẩm.

CEO Lovable (Summer Labs)

Công nghệ tốt nhưng yêu cầu đối tác phong làm việc của user hiện có để tích hợp.
Kết quả: Khó được thị trường chấp nhận.

Bài học

Đừng xây AI rồi mong mọi người tự tìm cách dùng. Hãy bắt đầu từ trải nghiệm end-to-end và xác định đúng điểm AI giải quyết vấn đề người dùng thực sự quan tâm.

Câu hỏi không còn là **HOW to build** — mà là **WHAT TO BUILD**

Tìm bài toán AI ở đâu?

4 Lenses — Scan from self

🔄

Lặp lại

Việc gì tôi/team làm đi làm lại mỗi ngày/tuần?

⏰

Tốn thời gian

Việc gì mất nhiều thời gian hơn lẽ ra nên mất?

🤖

AI có thể tốt hơn

Sản phẩm nào tôi dùng mà AI có thể cải thiện?

💬

Pain từ người khác

Đồng nghiệp/bạn bè hay phàn nàn gì?

Lab hôm nay bắt đầu bằng scan from self — dùng 4 lenses này để quét xung quanh mình trước khi chọn bài toán.

4 anti-patterns làm team đốt tiền vào AI sai chỗ

- ☐ **Trend-first:** Thay đổi theo trend “agent” trước khi rõ actor, workflow, metric
- ☐ **No baseline:** Không có rule/manual baseline để so sánh nhưng vẫn build AI trước
- ☐ **No eval path:** Có demo đẹp nhưng không có bộ test, không biết khi nào đủ tốt để deploy
- ☐ **No owner of failure:** Không rõ ai review output sai, ai rollback, ai chịu compliance

☑ **Nguyên tắc:** Dùng AI khi nó tạo giá trị hơn cách đơn giản hơn, không phải vì nó nghe hiện đại hơn

Nhiều team nghĩ mình đang build AI. Thực ra họ đang build mơ hồ.

Discovery interview: 5 câu hỏi nên hỏi stakeholder

- **Pain point là gì?** Xảy ra bao nhiêu lần / ngày / tuần?
- **Workflow hiện tại ra sao?** Tool nào, bước nào, ai hand-off cho ai?
- **Cost của vấn đề là gì?** Mất bao nhiêu phút, tiền, SLA, hay conversion?
- **Nếu AI sai thì sao?** Điểm nào cần HITL, approval, hoặc chỉ để suggest?
- **Ai sẽ nói YES?** Metric nào và risk nào quyết định việc đầu tư?

Lưu ý: Nếu stakeholder không mô tả được workflow hiện tại và failure cost, team đang đề xuất solution trong sương mù.

Có nên dùng AI không và dùng mức nào?

Khi nào AI đáng để làm?

AI HỢP KHI NÀO

- Tác vụ lặp lại nhưng biến thể vừa phải
- Cần tổng hợp / search
- Nhiều bước + nhiều tool
- *Deterministic thì rule có thể tốt hơn*

VÌ SAO DOANH NGHIỆP ĐẦU TƯ

01 Sống còn — không dùng AI → bị đối thủ vượt

02 Hiệu quả — giảm chi phí, tăng tốc, cải thiện throughput

03 Khám phá — đầu tư để học, tránh tụt hậu, tìm cơ hội mới

Lý do áp dụng AI sẽ quyết định cách build, mức tự động hóa và mức đầu tư.

Tự xây dựng hay mua giải pháp?

Hai góc nhìn bổ sung nhau

GÓC NHÌN 1 — CHIP HUYEN, AI ENGINEERING (2025)

In-house (Build)

Khi AI là **lợi thế cốt lõi** và yếu tố **sống còn**

Mua / SaaS (Buy)

Khi AI chủ yếu là **productivity layer**

GÓC NHÌN 2 — MIT CISR (VAN DER MEULEN & WIXOM, 2025)

Buy

- Off-the-shelf, vendor duy trì
- Nhanh, ít differentiation
- Phụ thuộc vendor roadmap

Boost

- Mua model + enhance bằng data riêng
- Fine-tune hoặc RAG
- Cần data governance tốt

Build

- Tự xây custom model
- Control cao nhất, đắt nhất
- Cần team AI mạnh

Thực tế: Hầu hết team đang ở giữa — **Boost** (RAG / fine-tune). Không phải lúc nào cũng cần build from scratch.

Thiết lập kỳ vọng

Đo gì để biết AI có đáng triển khai?

Trước khi build, phải biết thế nào là "đủ tốt"

1 — TÁC ĐỘNG KINH DOANH

Question: AI tạo giá trị gì cho doanh nghiệp?

Đo bằng:

✓ % tác vụ / tin nhắn tự động hóa
✓ khả năng xử lý tăng thêm
✓ tốc độ phản hồi cải thiện
✓ thời gian lao động tiết kiệm

2 — SỰ HÀI LÒNG KHÁCH HÀNG

Question: Người dùng có thật sự thấy tốt hơn không?

Đo bằng:

✓ CSAT (Điểm hài lòng khách hàng)
✓ phản hồi trực tiếp
✓ hành vi sử dụng / bỏ giữa chừng / phải retry

3 — NGƯỠNG HỮU DỤNG

Question: Tới mức nào thì sản phẩm đủ tốt để ship?

Đo bằng:

Chất lượng: đầu ra có đúng và hữu ích không?
Độ trễ: TTFT, TPOT, thời gian phản hồi
Chi phí: mỗi request tốn bao nhiêu?

✍️ Bài tập cá nhân

Chọn **1 pain point** bạn viết lúc đầu. Thử trả lời:

1. Tác động kinh doanh — ai đang mất gì?

→ Mất bao nhiêu giờ/tuần? Bao nhiêu người bị ảnh hưởng?

2. Người dùng — ai đang đau? Họ nói gì?

→ Họ đang xử lý tạm bằng cách nào? Phàn nàn gì?

3. Đo bằng gì để biết "đủ tốt"?

→ Thời gian giảm bao nhiêu %? Số lỗi giảm? Con số cụ thể?

Gửi lên Discord · 5 phút

Từ demo đến production

Vì sao AI mất thời gian thật sự?

DEMO

1 cuối tuần

✓ Verify giả thuyết
✓ Align stakeholder
✓ Test nhanh UI/UX

PRODUCTION

Nhiều tháng → năm

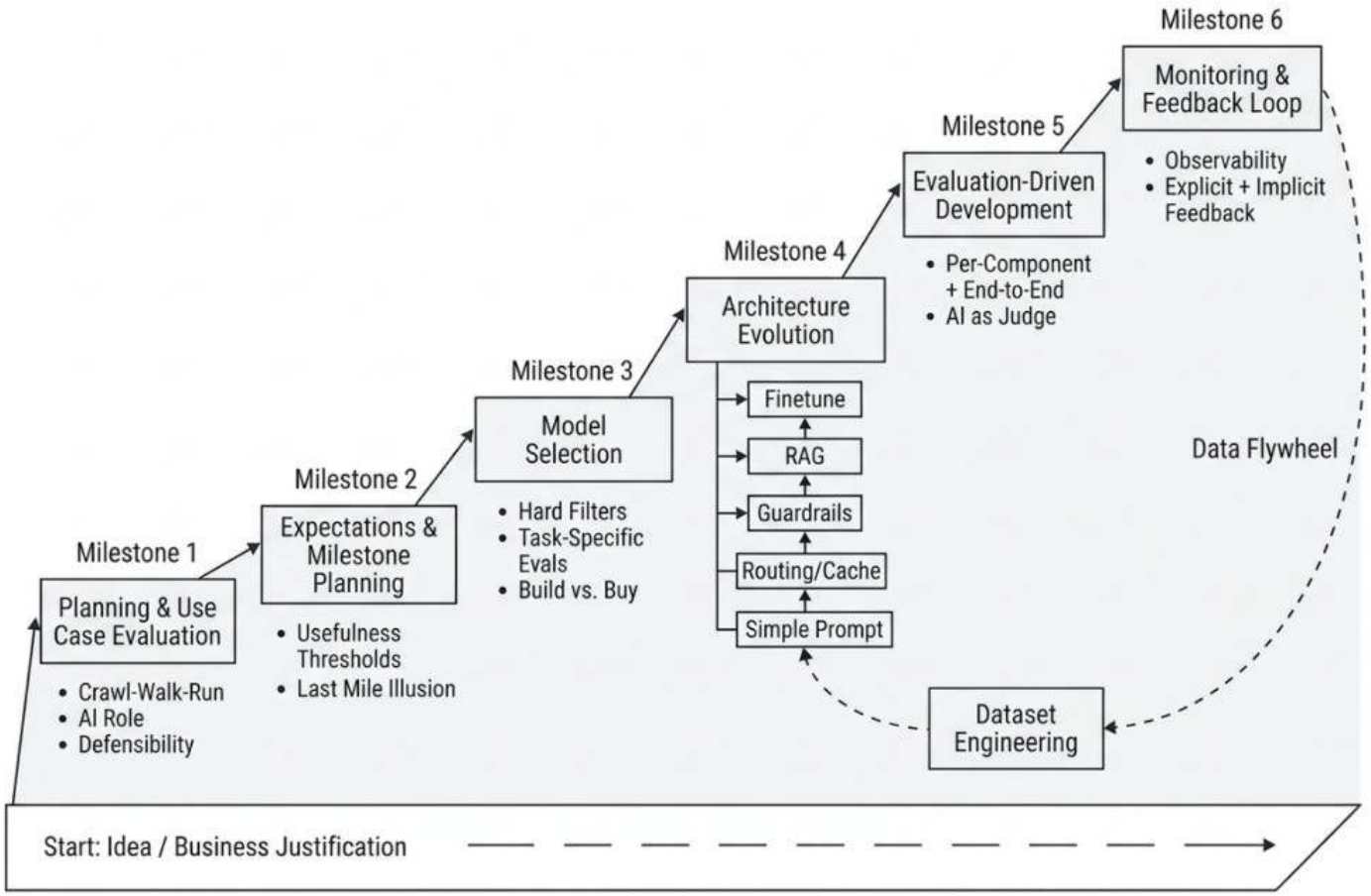
✕ Edge cases, guardrails
✕ Hallucination
✕ Giành từng 1% chất lượng

80% → 95% là đoạn đau nhất

LinkedIn: 80% trải nghiệm trong 1 tháng, mất thêm 4 tháng để nhích lên 95%

AI Product Lifecycle

6 milestones từ ý tưởng đến vận hành



Data Flywheel: feedback từ production → cải thiện data → model → architecture

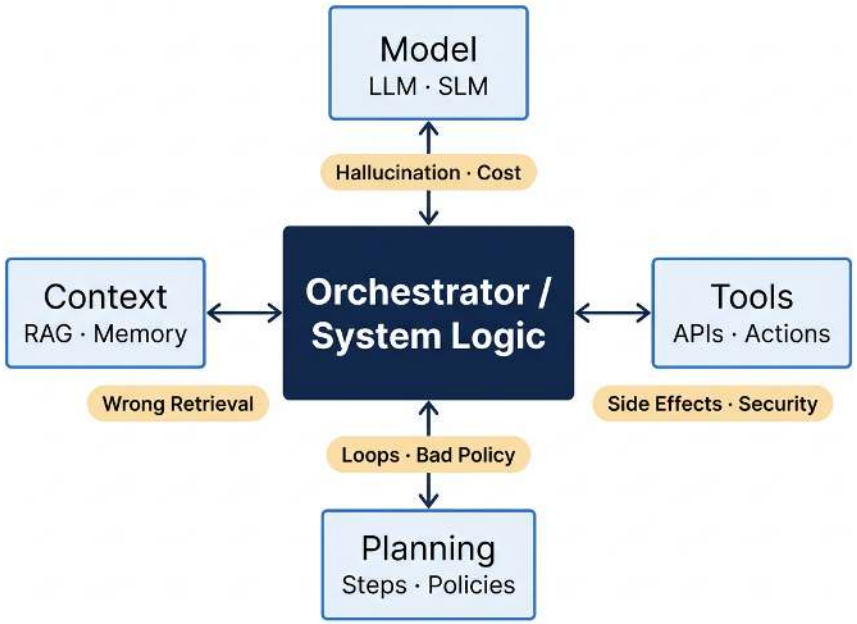
Vai trò của UX trong AI product

AI không cần hoàn hảo, nếu UX đỡ được chỗ nó yếu

Không chắc (low confidence)	→	**Xin user xác nhận trước khi thực hiện**
Risk cao (sai = hậu quả nghiêm trọng)	→	**Chỉ suggest, không auto-action**
Câu trả lời dài (quá tải thông tin)	→	**Chia option / card / summary cho user chọn**
Thiếu context (input mơ hồ)	→	**Hỏi lại đúng chỗ thay vì đoán sai**

AI Product = AI + UX. Dùng UX để hỗ trợ chỗ AI chưa đủ tốt.

AI System = Model + Context + Planning + Tools



Thành phần	Cần khi...	Failure mode	Control	Cost driver
Model	Task mở, cần suy luận / tạo văn bản	Hallucination, inconsistency	Eval set, guardrails, structured output	tokens / latency
Context	Cần dựa trên tài liệu / trạng thái ngoài	Wrong retrieval, stale context	Retrieval test, freshness, citations	storage / retrieval
Planning	Task nhiều bước, cần ra quyết định trung gian	Looping, over-planning	Step limit, policy, approvals	extra calls
Tools	Cần đọc / ghi hệ thống ngoài	Side effects, prompt injection	sandbox, allowlist, HITL	API calls / failures

Framework chọn Rule / Workflow / Agent

04

Bắt đầu đơn giản nhất — chỉ tăng phức tạp khi bài toán thật sự cần

Ba mức giải pháp: Rule / Workflow / Agent

Rule / Script

- Đầu vào ổn định, ít thay đổi
- Logic viết được thành if/else
- Cần kết quả luôn đúng 100%
- Quy định pháp lý / tuần thủ chặt

Ví dụ: Tính thuế, chặn email spam theo từ khóa, auto-reply theo template

LLM Feature / Workflow

- Đầu vào đa dạng, không viết hết rule được
- Đầu ra cần linh hoạt (tóm tắt, dịch, phân loại)
- Có cách đo chất lượng
- Người có thể kiểm tra trước khi gửi

Ví dụ: Tóm tắt email, chatbot FAQ, phân loại ticket hỗ trợ

Agent

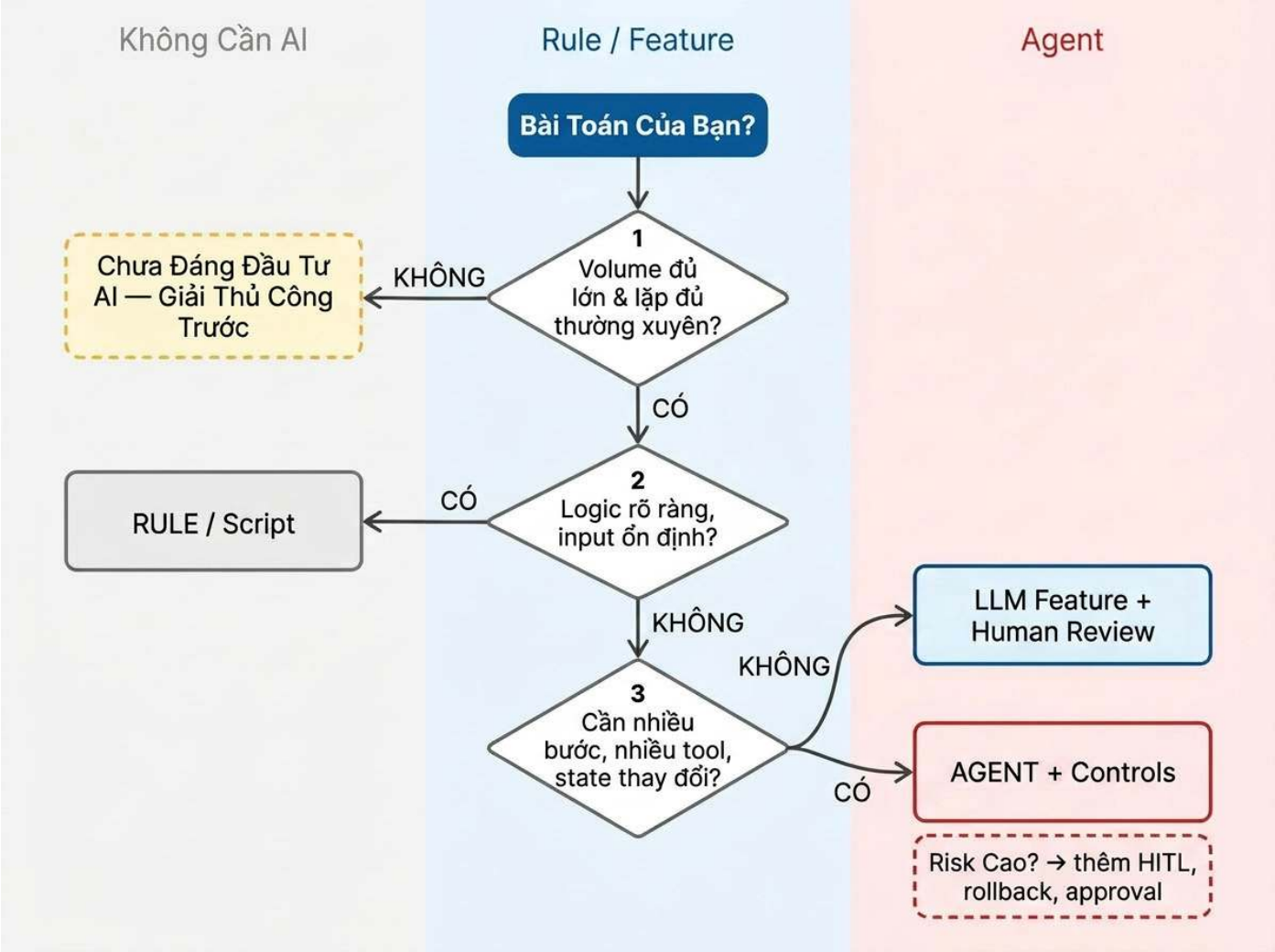
- Nhiều bước, dùng nhiều công cụ
- Tình huống thay đổi liên tục
- Cần tự ra quyết định giữa các bước
- Có kiểm soát rủi ro rõ ràng

Ví dụ: Agent nghiên cứu thị trường, coding agent sửa nhiều file

Thứ tự ưu tiên thực dụng: bắt đầu từ bên trái, chỉ đi sang bên phải khi giá trị tăng hơn độ phức tạp.

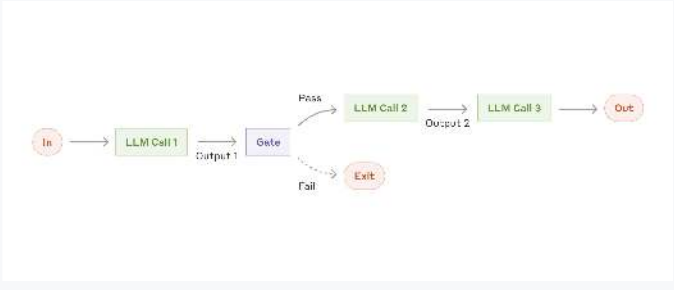
Decision tree: chọn mức giải pháp

Từ bài toán của bạn → Rule, LLM Feature, hay Agent?



Workflow patterns — đủ cho hầu hết bài toán

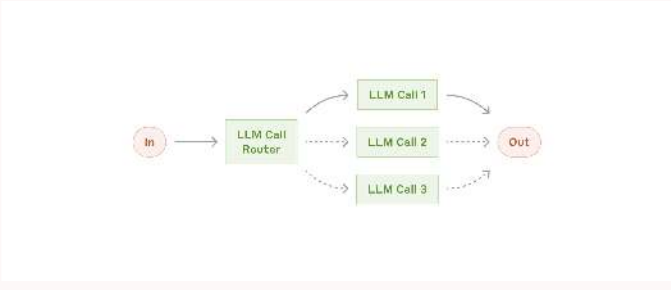
Nguồn: Anthropic — Building Effective Agents (2024)



1. Prompt Chaining

Chia task thành chuỗi bước tuần tự. Có **gate** kiểm tra giữa các bước.

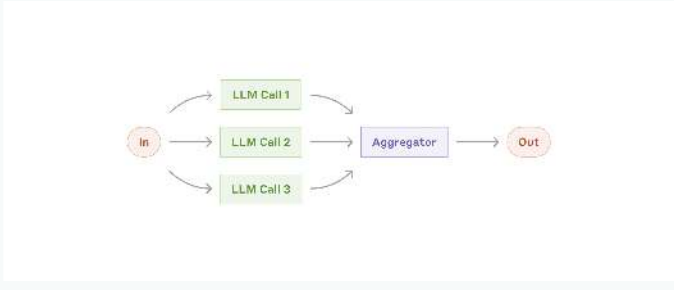
VD: Viết outline → check → viết bài



2. Routing

Phân loại input → đưa vào nhánh chuyên biệt. Tối ưu từng loại riêng.

VD: CS query → FAQ / refund / kỹ thuật



3. Parallelization

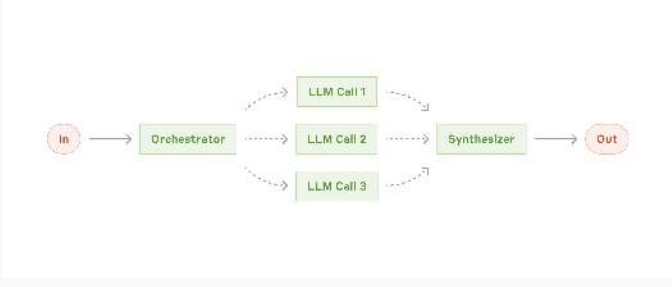
Chạy song song rồi tổng hợp (**sectioning**), hoặc chạy nhiều lần lấy **vote**.

VD: Guardrail + response đồng thời

Nguyên tắc Anthropic: “Start with the simplest solution possible, only increase complexity when needed.” — Hầu hết bài toán thực tế chỉ cần 3 patterns này.

Khi nào cần phức tạp hơn?

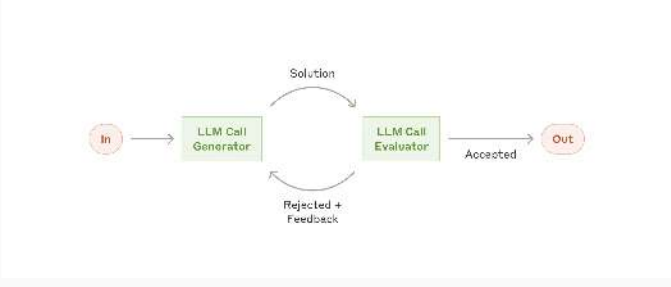
Orchestrator-Workers, Evaluator-Optimizer, và Agent



4. Orchestrator-Workers

1 LLM phân việc động cho workers. **Subtasks không biết trước**.

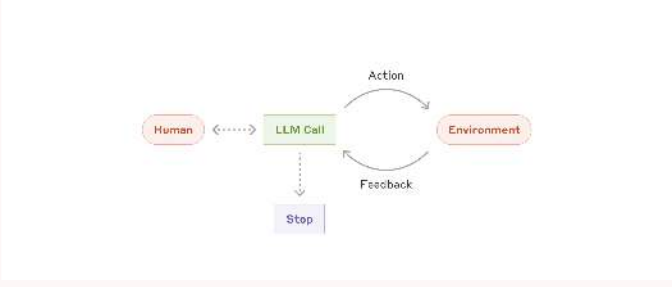
VD: Coding agent sửa nhiều file



5. Evaluator-Optimizer

1 LLM tạo, 1 LLM đánh giá → **lặp cho đến khi đạt**.

VD: Dịch văn học → review → sửa



Agent

LLM tự lập kế hoạch + gọi tools + iterate. **Autonomous loop**.

VD: SWE-bench, computer use

Chi phí cao hơn, lỗi cộng dồn. Chỉ dùng khi không predict được subtasks và cần tool autonomy. "Agents' autonomy makes them ideal for scaling tasks in **trusted environments**."

AI readiness checklist - 5 câu hỏi nhanh

- ✓ **Value:** Bài toán xảy ra thường xuyên và đang tốn thời gian / tiền / SLA thật sự?
- ✓ **Baseline:** Hiện tại đã có manual hoặc workflow baseline để so sánh?
- ✓ **Eval:** Có metric, sample cases, hoặc logs để đánh giá reproducible?
- ✓ **Tolerance:** Sai số hữu hạn có chấp nhận được, hoặc có HITL ở điểm quan trọng?
- ✓ **Operations:** Có owner, rollback, logging, và policy cho output sai?
- × Dưới 3 câu YES: dừng lại và làm rõ problem / workflow trước khi đầu tư AI

Thiếu baseline hoặc thiếu eval = chưa biết AI đang tốt hơn cái gì

Lưu ý: AI "Jagged Frontier" — ranh giới AI gồ ghề, khó đoán. Framework là starting point, không phải kết luận. Phải test thực tế. (Dell'Acqua, Mollick et al. 2023)

Gate criteria: đi tiếp khi nào, dừng lại khi nào?

Rules of ML + practical AI product heuristics

Giai đoạn	Go nếu...	No-Go nếu...
Problem Scoping	Actor, workflow, pain, metric đã rõ	Problem mơ hồ, mô tả solution trước problem
Data Readiness	Có data / logs / docs / SME support để eval	Không có nguồn dữ liệu hoặc dữ liệu quá lệch
Baseline / Model	Heuristic, workflow, hoặc baseline người được xác lập	Chưa biết AI cần tốt hơn cái gì
Build & Eval	Có bộ test, task-level eval, latency/cost budget	Chỉ có demo thủ công, không có reproducible eval
Deploy Controls	Có HITL, logging, approval, rollback, owner of failure	Không rõ ai duyệt output, ai dừng hệ thống khi sai

Key takeaways:

- Không có metric → không có gate
- Không có baseline → chưa biết tốt hơn gì
- Không có eval → mới chỉ là demo
- Không có rollback → deploy là đánh bạc

Problem Statement

Từ ý tưởng mơ hồ đến bài toán có metric, boundary, và đường eval

05

Khung mô tả bài toán cho AI system

Viết xong phải suy ra được: cách kiểm tra, chỉ số đo, và ranh giới hệ thống

Actor (Người thực hiện)	Ai đang làm việc này hằng ngày?
Workflow (Quy trình hiện tại)	Hiện tại họ xử lý qua những bước nào, dùng công cụ gì?
Bottleneck (Điểm nghẽn)	Bước nào chậm, hay lỗi, không nhất quán, hoặc cần tổng hợp quá nhiều?
Impact (Mức ảnh hưởng)	Tổn thất đo bằng thời gian, chi phí, cam kết chất lượng, tỷ lệ lỗi, hay chuyển đổi?
Success Metric (Chỉ số thành công)	Khi nào được coi là thành công? Mức ngưỡng cụ thể là bao nhiêu?
Boundary (Ranh giới hệ thống)	AI được phép làm gì, không được làm gì, và điểm nào cần người duyệt?

Nhớ

Ví dụ: Giảng viên chấm bài Lab

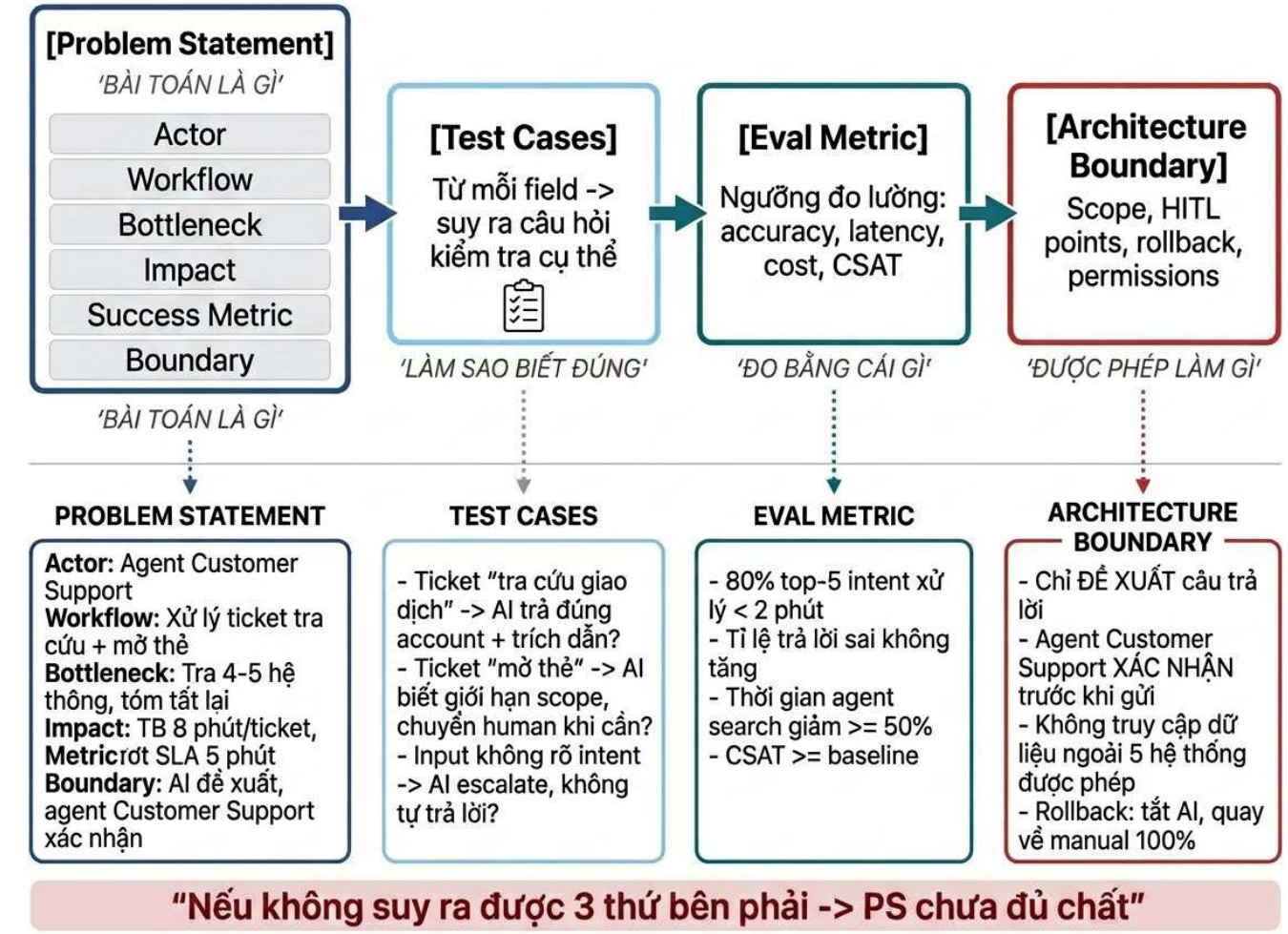
Điền 6 ô — từ mô tả chung thành bài toán rõ ràng

Actor (Người thực hiện)	Giảng viên + Lab Coach — 10 người chấm cho 500 sinh viên, lặp mỗi buổi lab
Workflow (Quy trình hiện tại)	Nhận bài → Đọc từng bài → Đối chiếu rubric → Ghi điểm + nhận xét → Tổng hợp kết quả
Bottleneck (Điểm nghẽn)	Bước 2 & 3: đọc + đối chiếu rubric cho 50 bài/người, tốn ~3h/buổi
Impact (Mức ảnh hưởng)	10 người × 3h = 30 giờ/buổi lab. Trả kết quả chậm → SV không kịp sửa cho bài tiếp
Success Metric (Chỉ số thành công)	Giảm từ 3h → dưới 1h/người. Độ khớp với điểm giảng viên chấm tay ≥ 85%
Boundary (Ranh giới hệ thống)	AI chấm sơ bộ + gợi ý nhận xét. Giảng viên duyệt điểm cuối và sửa nhận xét trước khi trả SV.

Suy ra được gì?

- **Cách kiểm tra:** Lấy 50 bài, so điểm AI với điểm giảng viên chấm tay → đo độ khớp

Từ problem statement sang eval plan



> Problem Statement tốt là cầu nối giữa bài toán kinh doanh và bộ eval kỹ thuật.

Chọn đúng metric: North Star & input/output

Nguồn: Lenny Rachitsky (a16z) — North Star Metric Framework

Câu hỏi: metric nào nếu tăng hôm nay sẽ tăng tốc flywheel kinh doanh nhất?

Revenue	Customer Growth	Consumption	Engagement	Growth Efficiency	User Experience
ARR, GMV	Paid users, market share	Messages sent, nights booked	MAU, DAU	LTV/CAC, margins	NPS, CSAT

Output Metric (North Star)

Kết quả cuối — team không trực tiếp control

Airbnb: "nights booked"

Input Metrics (Levers)

Cái team làm hằng ngày để đẩy output lên

Airbnb: conversion rate, supply of homes, site traffic

PS field "Success Metric": cần cả output metric (thành công = gì?) và input metric (team làm gì hằng ngày?). "Cải thiện hiệu suất" không phải metric.

Quyết định: Làm / Chưa làm / Không làm

✅ Go — Làm

Bài toán rõ

Có cách so sánh trước/sau

Có cách đo chất lượng

Biết ai chịu trách nhiệm khi sai

VD: Chấm bài lab — có rubric, có điểm GV đối chiếu, GV duyệt cuối

⏸️ Not Yet — Chưa làm

Vấn đề có thật

Nhưng thiếu dữ liệu

Hoặc chưa rõ đo bằng gì

Hoặc chưa hiểu quy trình đủ sâu

VD: Muốn AI hỗ trợ SV — nhưng chưa biết SV kẹt ở đâu, cần gom data trước

🚫 No-Go — Không làm

Luật / script đã đủ tốt

Sai thì hậu quả quá lớn

Hoặc chi phí thay đổi cao hơn giá trị

VD: Tính điểm GPA — công thức cố định, không cần AI

Lab hôm nay không bắt buộc ra kết luận "Làm". Kết luận tốt có thể là "Chưa làm" hoặc "Không làm" — nếu lý do rõ ràng.

Bài tập nhóm: Reverse-engineer sản phẩm AI

Viết lại bài toán mà team đó đã giải — dùng khung 6 ô | **15 phút · Nộp lên Discord**

Chọn 1 sản phẩm AI bạn hay dùng. Viết lại Problem Statement mà team đó đã giải.

1. Cả nhóm chọn 1 sản phẩm AI (2 phút)
2. Dùng thử / đọc review / thảo luận (3 phút)
3. Cùng viết Problem Statement 6 ô (7 phút)
4. Nộp lên Discord (3 phút)

Ví dụ: Grammarly

Actor	Người viết tiếng Anh không phải native, ~30 triệu user
Workflow	Viết → tự đọc lại → sửa → gửi → vẫn có lỗi
Bottleneck	"Tự đọc lại" — 10-15 phút, vẫn sót lỗi ngữ pháp + tone
Impact	Email có lỗi → mất chuyên nghiệp, mất deal
Metric	Giảm lỗi 80%, review từ 15 phút → 2 phút
Boundary	Chỉ gợi ý, không tự sửa. Không viết hộ nội dung.

Tự hỏi sau khi viết: Bạn có phải dùng thử mới viết được không? Có phải đọc review mới biết pain point không? — Đó là research. Không research thì PS chỉ là đoán.

5 điều mang về từ ngày 2

1. **Hỏi trước, làm sau.**
   Stakeholder nói “muốn chatbot” — đó là triệu chứng, không phải bài toán. Tìm vấn đề thật trước.

2. **Đơn giản trước, phức tạp sau.**
   Rule → Workflow → Agent. Đừng dùng agent khi một script 10 dòng là đủ.

3. **Không đo được = không biết có tốt không.**
   Chỉ số phải có con số cụ thể: “giảm từ 3h → 1h”, không phải “nhanh hơn”.

4. **AI làm nháp, người duyệt cuối.**
   Ranh giới rõ ràng: AI được làm gì, không được làm gì, ai chịu trách nhiệm.

5. **“Chưa làm” không phải thất bại.**
   Biết mình chưa đủ data để build — đó là quyết định trưởng thành nhất.

Thực hành

08

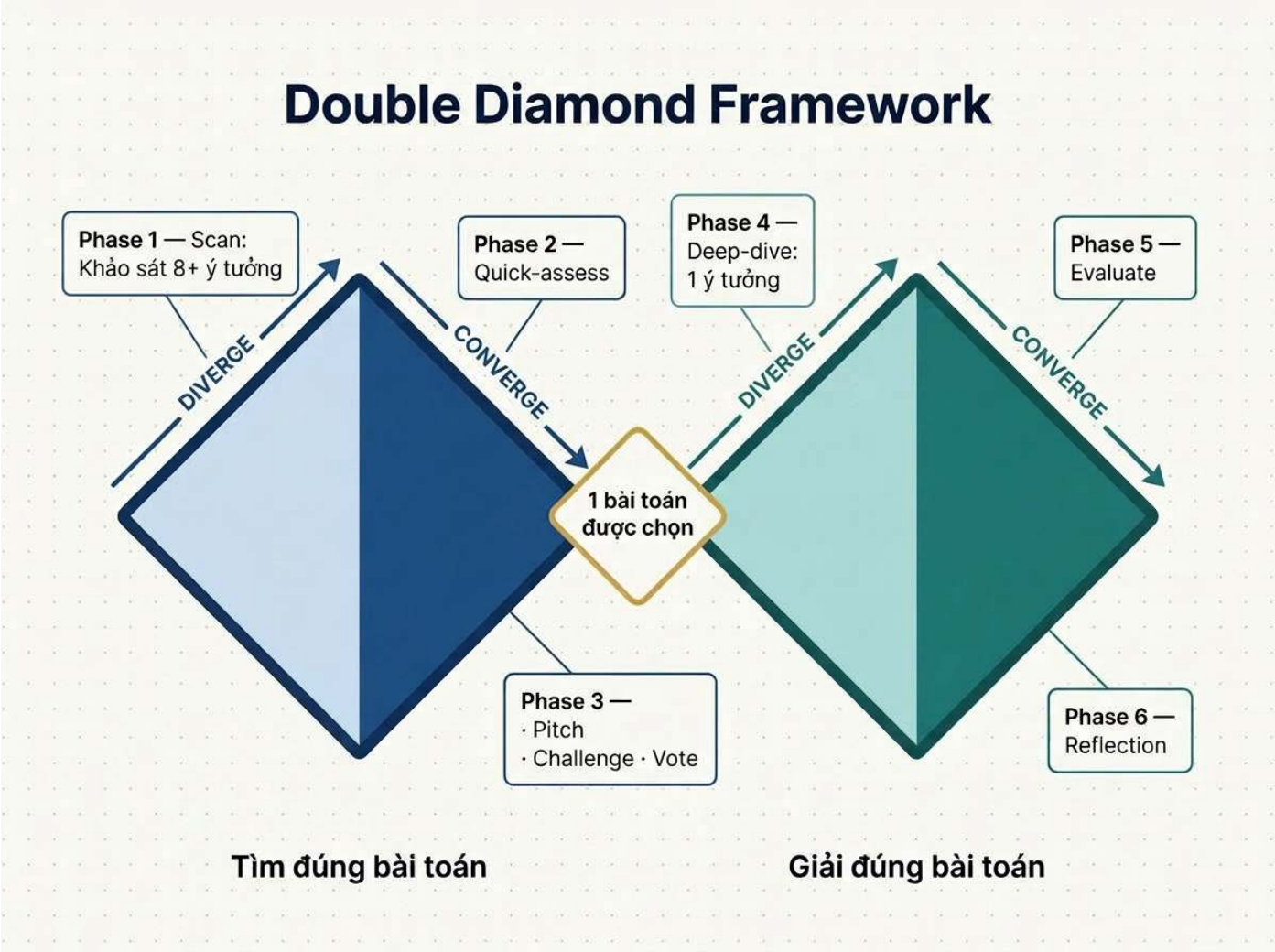
Lab 2: Chọn use case, viết PS, và ra quyết định go/no-go

Tổng quan lab 2 — deliverables & AI rules

PHASE	Phase 0	Phase 1	Phase 2	Break	Phase 3	Phase 4	Break	Phase 5	Phase 6
	15 min	20 min	40 min	10 min	30 min	85 min	10 min	20 min	10 min
DELIVERABLE	Hiểu flow mẫu	8+ Ideas	3 Quick Problem Cards		1 bài toán được chọn	Workflow + PS + Research + Future Flow		Go / Not Yet / No-Go	AI Support Log
AI USAGE	AI OK ✅	AI OK ✅	AI OK ✅		NO AI ⛔	AI OK ✅		NO AI ⛔	NO AI ⛔

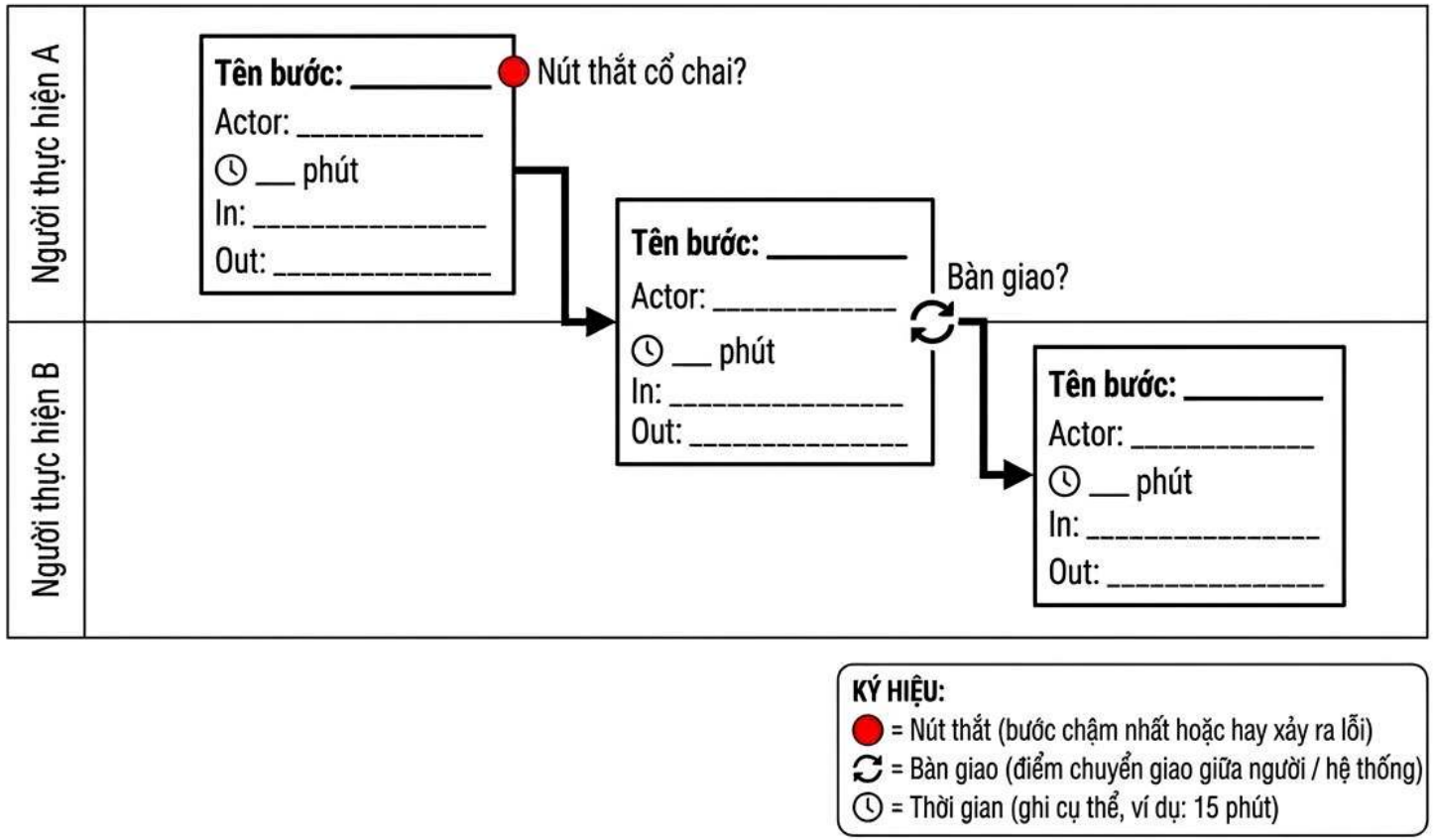
Phase 1 + 2

Hoạt động cá nhân



Hướng dẫn vẽ workflow diagram

Dùng cho Phase 4 — Deep-dive: Current-State & Future-State



- ☑ Tối thiểu 5-8 bước (đừng gộp quá)
- ☑ Each step has a specific time duration
- ☑ Ít nhất 1 nút thắt được đánh dấu
- ☑ Mỗi bước có thời gian cụ thể
- ☑ Ghi tổng thời gian của workflow
- ☑ VẼ trên giấy, không gõ text

Worked example: weekly report — trước và sau AI

Chấm bài Lab - 6 bước, 3h → 5 bước, 50 phút

CURRENT STATE 3h (mỗi Lab Coach, 50 bài)

BOTTLENECK 50'

① Nhận bài từ LMS 10 phút — Coach

② Đọc từng bài 60 phút — Coach

③ Đối chiếu rubric 40 phút — Coach

④ Ghi điểm + viết nhận xét 50 phút — Coach

⑤ Tổng hợp bảng điểm 10 phút — Coach

⑥ Gửi kết quả cho SV 10 phút — Coach

0–3h → 0–3h

--> 10 Coach × 3h → 10 Coach × 3h = 30 giờ/buổi lab. SV nhận kết quả chậm, không kịp sửa cho bài tiếp.

FUTURE STATE 50 phút

Boundary

① Auto-pull bài từ LMS 2'

② AI chấm sơ bộ theo rubric 5'

③ AI gợi ý nhận xét 3'

④ GV/Coach duyệt điểm + sửa nhận xét 35'

⑤ Coach gửi kết quả 5'

0–50 phút → 0–50

--> **Fallback:** AI chấm sai → Coach sửa lại thủ công (vẫn nhanh hơn viết từ đầu)

Tiêu chí	Trước	Sau	Thay đổi
Tổng thời gian	3h phút	50 phút	-72%
Bước thủ công	6/6 thủ công	2/5 thủ công	–
Bottleneck mới	Viết nhận xét	Duyệt nhận xét	nhẹ hơn

Phase 3 · Pitch – Challenge – Vote

Framework peer review trong nhóm · 30 phút

🎤 PITCH

2 phút / người

- Present bài toán của mình cho nhóm
- Dùng Quick Problem Card làm visual

⚡ CHALLENGE

3 câu hỏi chuẩn · 3 phút / người

- "Rule/script đủ chưa? Có thật sự cần AI không?"
- "Ngoài bạn, ai đau nữa? Bao nhiêu người?"
- "Metric đo được không? Có số cụ thể chưa?"

🗳️ VOTE

5–10 phút sau khi tất cả pitch

- Mỗi người 1 phiếu → chọn 1 bài đi tiếp
- Viết 1 dòng kill rationale cho bài bị loại

Quick problem card — ví dụ đã điền

Mỗi người điền 1 card trong Phase 2 · ~5 phút

Bài toán (1 câu)	Mất 3h/ngày để tổng hợp feedback từ 200+ email khách hàng thành báo cáo cho team sản phẩm
Ai đang đau?	Product Manager · ~50 PM trong công ty · lặp lại mỗi tuần
Workflow hiện tại (3-5 bước)	Nhận email → đọc từng email → tự tóm tắt → gộp thành doc → gửi team
Bước nào tốn nhất?	Bước 2 — đọc & tóm tắt (~2.5h/ngày · 150h/tuần × 50 PM)
AI có thể giúp ở bước nào?	Bước 2 — AI tóm tắt từng email, PM review + edit trước khi gửi
Đo thành công bằng gì? (có số!)	Giảm từ 3h → dưới 30 min/ngày · chất lượng báo cáo đồng đều hơn
Quick gut	☑ LLM — cần xử lý ngôn ngữ tự nhiên đa dạng, không thể rule-based

💡 **Nhận xét**

Bài toán rõ ràng, đo được, đủ scale (50 người × hàng tuần). AI phù hợp vì cần xử lý ngôn ngữ tự nhiên đa dạng.

Lab 2 — Deliverables — Mỗi người cần nộp gì?

Deliverables Example

LAB 2 · 4 giờ

Scan bài toán, filter, deep-dive, và quyết định go/no-go

1

Scan & Filter Log

Liệt kê 5+ bài toán (4 Lenses) → Quick Problem Card top 3 → Peer validate → Chọn 1

Cá nhân

2

Problem Deep-Dive

Current-state flow + PS + metrics + research + future-state flow + AI fit + Go/Not Yet/No-Go

Nhóm

3

Reflection Log

Ghi lại cách dùng AI trong quá trình lab: prompt, output, và chỗ phải sửa

Cá nhân